Long Polling

Because traditional polling is inefficient, the next improvement is **Long Polling**.

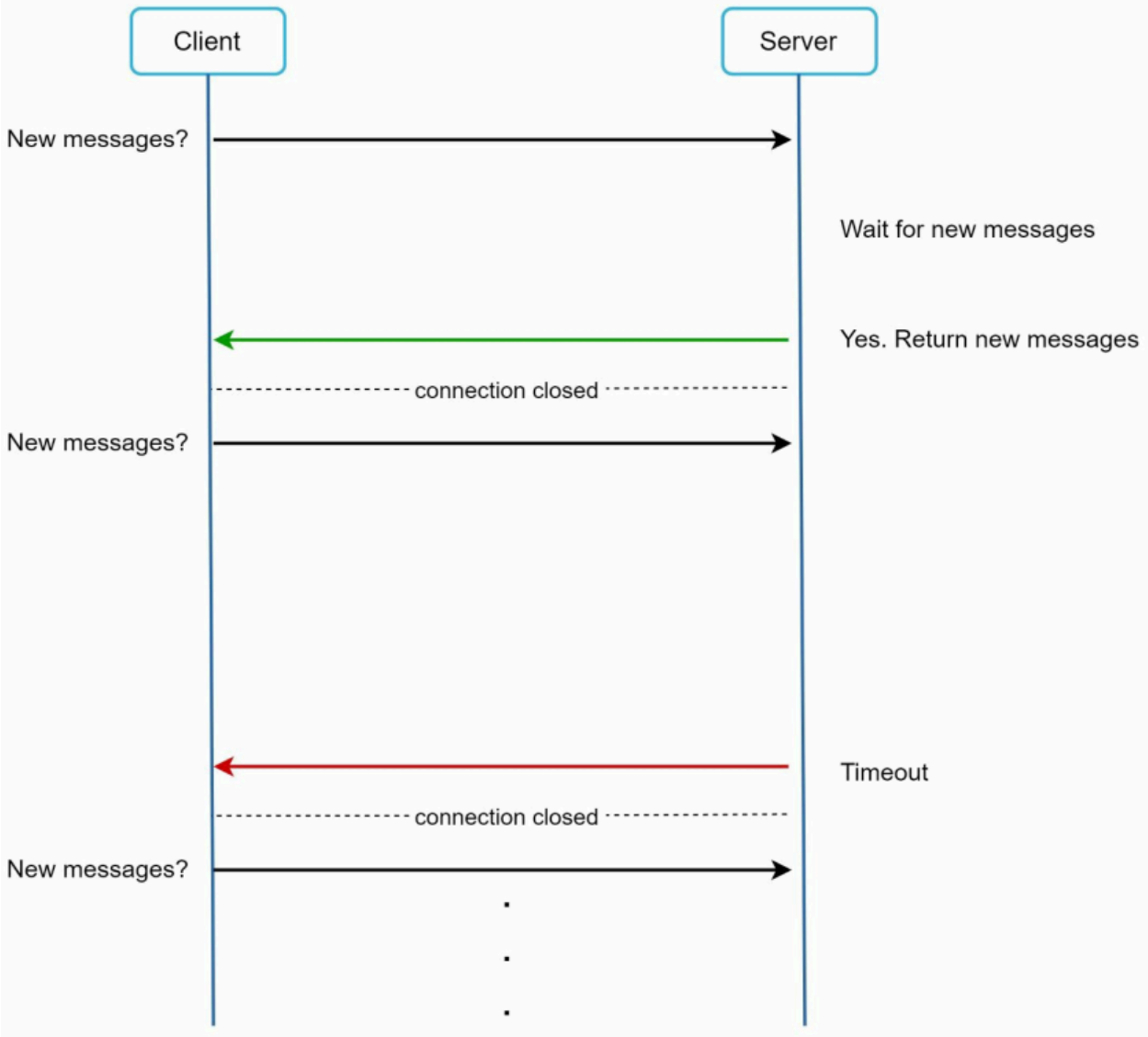
In long polling, the **client sends a request and keeps the HTTP connection open** until:

```
None
1. New data/message becomes available

OR

2. Request timeout is reached
```

Once the client receives a response, it **immediately sends another request**, repeating the cycle.



As shown in **Figure**, long polling reduces unnecessary empty requests compared to normal polling.

1. Core Idea

Instead of asking every few seconds:

```
None
Any message?

Any message?

Any message?
```

The client asks once:

```
None
Tell me when a message arrives.
```

The server waits until something happens.

2. How Long Polling Works

```
None
1. Client sends request to server

2. Server keeps request open

3. If message arrives → respond immediately

4. If timeout occurs → send empty response

5. Client instantly reconnects

6. Repeat
```

3. Example Flow

```
Client → GET /messages

(Server waits 25 sec...)


[After 8 sec new message arrives]


Server → "Hello"

Client receives response

Client immediately sends next request
```

If no message arrives:

```
Client → GET /messages

(Server waits 25 sec...)

Server → Timeout / No data

Client reconnects again
```

4. Diagram

```
None
Time →


Client      ----Request----> Server

            (connection open)


            <---New Message----


Client      ----New Request----> Server

            (connection open)


            <---Timeout----
```

5. Why Long Polling is Better than Polling

✔ Fewer Empty Requests

Short polling:

```
None
Every 5 sec request even if no message
```

Long polling:

```
None
One request waits until needed
```

✔ Lower Latency

As soon as data is available:

```
None
Server responds immediately
```

No need to wait for next polling interval.

✔ Easy with HTTP

Works using normal HTTP infrastructure.

6. Drawbacks of Long Polling

The original content mentions important issues:

1. Load Balancer / Stateless Server Problem

Sender and receiver may hit different servers.

Example:

```
None
User A connected to Server 1

User B sends message via Server 2
```

If Server 2 doesn't know User A's waiting connection, delivery becomes difficult.

Solution:

```
None
Shared message broker (Redis/Kafka)

Sticky sessions

Central pub-sub layer
```

2. Hard to Detect Disconnects

If user closes browser or loses internet:

```
None
Server may not know immediately
```

TCP timeout detection can be delayed.

3. Still Inefficient for Inactive Users

If user never chats:

```
None
Request waits → timeout

Reconnect

Wait again

Timeout again
```

Still creates repeated connections.

4. High Connection Usage

Millions of users holding open requests means:

- Many open sockets
- More memory usage
- More server threads/workers (depending on architecture)

7. Polling vs Long Polling

Feature	Polling	Long Polling
Requests Frequency	Fixed Interval	Only after response
Empty Responses	High	Lower
Real-time Speed	Medium	Better
Server Resource Usage	High	Moderate
Complexity	Easy	Moderate

8. Long Polling vs WebSocket

Feature	Long Polling	WebSocket
Connection Type	Repeated HTTP requests	Persistent TCP connection
Duplex Communication	No	Yes
Real-time Performance	Good	Excellent
Scalability	Medium	High
Best For	Legacy HTTP systems	Chat / gaming / live apps

9. Use Cases

Long polling is suitable for:

```
None
Chat apps (small-medium scale)

Notification systems

Stock alerts

Live dashboards

Legacy systems without WebSocket support
```

10. Interview Insight

If interviewer asks:

Why move from polling to long polling?

Answer:

```
None
Long polling reduces unnecessary repeated requests

and provides faster updates because the server responds

only when data is available or timeout occurs.
```

11. Why Modern Systems Prefer WebSockets

At massive scale:

```
None
Millions of reconnecting long-poll requests

become expensive.
```

So modern chat apps use:

```
None
WebSocket

SSE

MQTT

Push Notifications
```

12. One-Line Summary

Long polling improves traditional polling by keeping the HTTP request open until new data arrives or timeout occurs, reducing empty requests and improving real-time responsiveness.